

NaijaBizFind - Backend Developer Guide (MERN Stack)

This document outlines the backend requirements for **NaijaBizFind**, a local business directory.

Tech Stack

- **Node.js & Express:** API Server.
- **MongoDB & Mongoose:** Database.
- **Paystack:** Payment gateway for listing fees.
- **Cloudinary:** Media storage for shop images and certificates.
- **JWT:** Authentication for Admin and Business owners (optional).

Database Schemas (Mongoose)

1. Business Schema

```
{
  name: { type: String, required: true },
  category: { type: String, required: true },
  city: { type: String, required: true },
  address: { type: String, required: true },
  description: { type: String, required: true },
  phone: { type: String, required: true },
  whatsapp: { type: String },
  workingHours: {
    open: { type: String }, // e.g., "06:00"
    close: { type: String } // e.g., "17:00"
  },
  images: {
    shopPhoto: { type: String, required: true }, // Cloudinary URL
    certificate: { type: String } // Cloudinary URL (Optional)
  },
  plan: { type: String, enum: ['basic', 'featured'], default: 'basic' },
  isPaid: { type: Boolean, default: false },
  status: { type: String, enum: ['pending', 'approved', 'rejected'], default: 'pending' },
  createdAt: { type: Date, default: Date.now }
}
```

2. Transaction Schema

```
{
  businessId: { type: Schema.Types.ObjectId, ref: 'Business' },
  reference: { type: String, required: true }, // Paystack reference
  amount: { type: Number, required: true },
  status: { type: String },
  paidAt: { type: Date }
}
```

API Endpoints

Public

- `GET /api/businesses` : Fetch all approved and paid businesses. Support filtering by category/city.

- `GET /api/businesses/:id` : Fetch single business details.
- `POST /api/businesses/register` : Create a new business entry (status: pending, isPaid: false).

Payment (Paystack)

- `POST /api/payments/initialize` : Initialize Paystack transaction.
- `POST /api/payments/verify` : Verify transaction status and update `Business.isPaid = true`.
- `POST /api/payments/webhook` : Handle Paystack webhooks to ensure payment consistency.

Admin (Protected)

- `GET /api/admin/submissions` : List all `pending` businesses.
- `PUT /api/admin/approve/:id` : Update status to `approved`.
- `PUT /api/admin/reject/:id` : Update status to `rejected`.

💰 Paystack Integration Logic

1. User selects plan (**Basic: ₦5,000** or **Featured: ₦10,000**).
2. Frontend calls `/api/payments/initialize` with `email` and `amount`.
3. Backend returns Paystack payment URL.
4. After payment, Frontend sends the `reference` to `/api/payments/verify`.
5. Backend checks Paystack API:
 - If successful, find the Business document and set `isPaid: true`.

☁ Media Handling (Cloudinary)

- Use `multer` and `cloudinary` (or `multer-storage-cloudinary`).
- The `shopPhoto` is mandatory.
- The `certificate` is optional.
- Generate high-quality thumbnails for the `GET /api/businesses` list using Cloudinary transformations.

🔧 Admin Flow

1. A business is only visible on the site if `isPaid === true` AND `status === 'approved'`.
2. Admin dashboard should allow the developer to review the description and photos before clicking "Approve".

🔑 Environment Variables (.env)

```
PORT=5000
MONGODB_URI=your_mongodb_connection_string
PAYSTACK_SECRET_KEY=sk_test...
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
ADMIN_PASSWORD=...
```